01 / EXECUTIVE ASSESSMENT

Multiplayer billing incident

An unattended room service kept scheduling paid background work after players had disconnected. The service is now permanently retired. Its ThreeWood Durable Object namespace and all room records were deleted with owner authorization on November 7.

$42.10	$89.21	0 reported
Durable Object incident charges[1]	Entire supplied invoice	Room invocations after shutdown[2]

Direct answer: do we still have Durable Objects?

No for ThreeWood; yes for the shared account. The RoomObject namespace and all of its room data have now been deleted. Cloudflare lists only the two unrelated SofaPaint UsageBudget namespaces. Those applications were left intact.

What is confirmed now

- **Solo game:** served by Cloudflare Pages project `threewood`, with no Pages Functions in the production deployment.
- **Multiplayer:** disabled in code and configuration. The custom Worker domain is removed; workers.dev and preview URLs are disabled.
- **Namespace:** `threewood-rooms_RoomObject` is deleted. The production Worker has no bindings or Durable Object class export.
- **Other invoice spend:** $42.05 intentional Workers AI usage, $5 subscription and $0.06 R2 storage.

Assessment

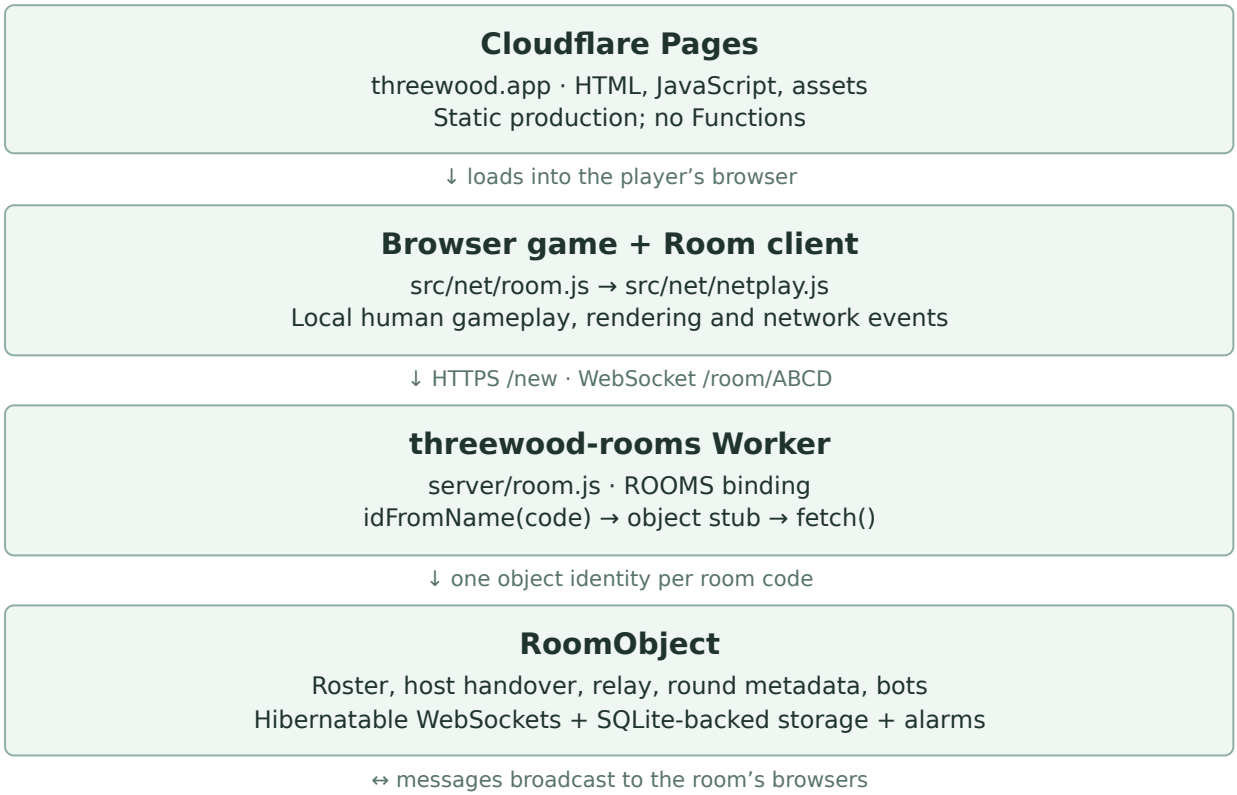
The sustained expense is strongly attributable to autonomous room alarms, not sustained player demand. The exact per-object execution history cannot be reconstructed because historical request traces were unavailable. Source inspection and retained metrics support the failure mechanism described in this report.

[1] Invoice lines are account-wide; namespace analytics overwhelmingly attribute the Durable Object activity to ThreeWood. [2] GraphQL returned no rows since 11:34 UTC / 19:34 Singapore, checked at 11:59 UTC. Reporting can lag. This is point-in-time evidence, not a continuous monitor or a per-object alarm audit.

02 / INTENDED ARCHITECTURE

How multiplayer is wired

The browser runs the golf game. A separate Worker routes each room to one stateful Durable Object, which coordinates players and runs bots.



Create / join

`/new` returns a random four-letter code; it does not reserve a room. The host connects with `?create=1`. Joining uses the same code without creation. The object accepts a WebSocket, then processes a `hello` message and returns welcome/roster data.

Cost boundary

Pages asset requests are free. Worker requests, object invocations, active object duration, database writes and stored data are separate metered resources. An alarm can incur cost even when nobody opens the website.

This diagram records the former enabled pipeline. Public multiplayer is retired. The production configuration has no ROOMS binding and includes a completed deletion migration; server/room.js is retained only as reference source and for regression tests.

03 / TECHNICAL ROOT CAUSE

Room lifecycle and failure mechanism

Normal responsibilities

Each room supports up to eight seats. Phones ping every five seconds. The server checks quiet sockets on a nominal 15-second sweep; disconnected seats should expire after three minutes. Empty rooms should be forgotten after 60 seconds. Bots step on alarms, nominally every 200 ms while playing.

Saved room state includes the roster, host, creation time and round metadata. The relay also keeps a bounded in-memory log for reconnects. That log and its message sequence are not durable across eviction; reconnect replay is therefore best effort, not a durable exactly-once guarantee. The server is a relay and bot host, not a fully authoritative anti-cheat simulation of human shots.

Two lifecycle deadlines lived in the wrong place

Defect	Original behavior	Consequence
Disconnected seat expiry	away() saved the away timestamp, then the sweep set lostAt only in memory. Expiry compared against lostAt.	Eviction could reset the removal countdown, allowing stale human seats to persist.
Empty-room cleanup	emptySince was set in memory and a wake-up scheduled 60 seconds later; the timestamp was not saved.	A cold constructor could forget the countdown and schedule another 60-second wait.
Autonomous bots	Bot scheduling did not require an attached human socket; no hard room lifetime existed.	Stale rounds could continue background work without a player.

The recurring-work loop

Alarm wakes room → room restores incomplete lifecycle state → stale seat or empty-room deadline fails to advance to deletion → work or cleanup is scheduled again → storage and compute are billed → repeat.

Hibernation/eviction discards JavaScript fields; it does not discard persisted room records or scheduled alarms. The design relied on process memory to decide when persistent work should stop. The ability to sleep between events did not bound the number of future events.

What we can and cannot prove

Both deadline defects are present in the retrieved deployed source. Metrics identify alarm traffic continuing without recorded sockets. This strongly supports the combined mechanism. Missing historical traces mean we cannot determine the exact share of cost caused by each defect, which object first entered the loop, or whether a specific eviction triggered each recurrence.

04 / FINANCIAL RECONCILIATION

Where every dollar went

The full invoice was $89.21. The unexpected Durable Object subtotal was $42.10. Nearly half the bill was intentional image generation, confirmed by the owner.



Invoice component	Charge	Disposition
Durable Objects: writes + duration + requests	$42.10	Unexpected; room service parked
Workers AI	$42.05	Owner confirmed intentional Flux image generation
Workers Paid	$5.00	Shared account subscription; retained
R2 storage	$0.06	Stored data; retained
Total	$89.21	All nonzero lines reconciled

Invoice scope and rounding

Invoice **IN-87619432**, issued November 7, covers metered usage October 7–November 6. The $5 subscription covers November 7–December 6. It lists 83,378,322 excess object requests, 16,401,872 excess written rows and 720,595 excess GB-seconds. Included allocations and billing-unit rounding explain why raw usage multiplied by the displayed unit price does not always equal the charged line exactly. [1]

Other charges were checked

All other lines on the supplied seven-page invoice are $0.00: ordinary Workers requests/CPU, KV, D1, logs, queues, containers, Images and the other listed products. A zero-dollar “Enabled” product is not an additional nonzero subscription on this invoice.

AI analytics show 443 Flux 2 Dev REST calls and about 4.127 million neurons during the billing period; voice-model usage was negligible. Latest recorded AI calls were November 4 at 04:00 UTC. The client or credential that made the REST calls was not identified; no AI credentials were revoked.

The invoice is the source of truth for amounts. GraphQL figures elsewhere are operational aggregates and may be adaptively sampled. Account billing APIs returned 403, so separate invoices or unrelated subscriptions were not exhaustively excluded.

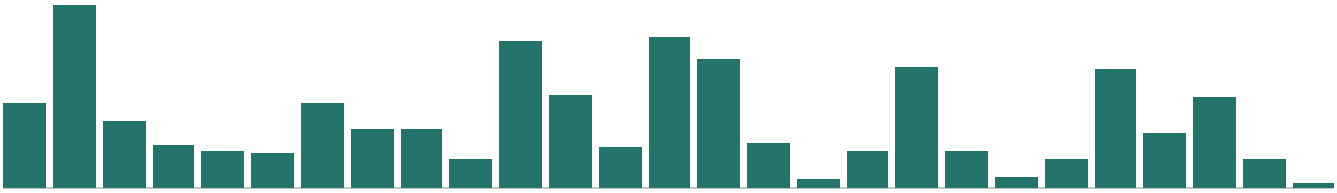
05 / IMPACT AND DETECTION

Evidence: alarms, not an audience

Room alarm invocations per day scale: 0–3.5 million



Zone page views per day scale: 0–2,000



*November 7 is partial, through shutdown. UTC dates. Different vertical scales; charts are intended to compare timing, not bar heights between panels.

Metric / observation	Reported result
Oct 12–Nov 7 room events	86,032,282 alarms; 130 HTTP; 13,904 hibernation events
Room database work	67,695,359 rows written; 14,453,307 rows read
Nov 6 contrast	3,433,622 alarms versus 9 outer Worker requests
Recorded connections	Maximum active WebSockets was zero on each day Oct 16 onward
Nov 6 onward object grouping	102 returned objects; only alarm events in that interval
Zone traffic Oct 12–Nov 7	18,158 page views; 29,794 HTTP requests

What the “10k page views” notification meant

It was a Cloudflare traffic milestone, not a spending limit or a count of players. Traffic included WordPress probes, unrelated asset paths and a self-identifying scanner. Human traffic may also be present. Solo play happens entirely in the browser, so the audited server metrics cannot measure round starts, completed holes or return visits.

Detection gap

The bill and owner-provided dashboard screenshot triggered investigation. There was no verified room-budget shutdown, lifecycle alert or monitor for “alarm work with no players.” Historical invocation logs/stacks were inaccessible; original settings did not show observability enabled and Logpush was off.

06 / TIMELINE AND DEPLOYED CONTROLS

Response and validation

Time / period	Event and confidence
Oct 12, 2026	Worker creation date; retained metrics already show alarm activity that day. Exact first defective invocation is unknown.
Oct 12–15	Some HTTP/WebSocket-related activity is recorded; it may include development testing.
Oct 16–Nov 6	Recurring alarms continue while periodic metrics report no active sockets.
Nov 7, 11:33 UTC 19:33 Singapore	Shutdown deployment reaches production; invocations fall during this minute.
Nov 7, from 11:34 UTC	Subsequent room-namespace queries return no invocation rows.
Nov 7, 11:34–11:59 UTC approximate response window	Lifecycle fixes, regression coverage, route removal, invoice investigation and hosting verification completed.
Nov 7, 11:59 UTC 19:59 Singapore	Fresh API verification confirms retained namespaces, empty Worker-domain list and disabled development/preview endpoints.

Initial containment controls (before retirement)

- **Fail closed:** ROOMS_ENABLED must be exactly “true.” Missing or false configuration stops ingress and object work.
- **Existing work:** constructors, alarms and messages cancel alarms and close sockets when disabled; close/error cleanup cannot rearm disabled rooms.
- **Durable deadlines:** emptySince is persisted; disconnected seats use persisted away timestamps.
- **Bounded work:** two-hour maximum room age checked on events; bot ticking requires an attached identified socket.
- **Cleanup:** deleteAlarm() is explicit before deleting stored room state. The compatibility date is January 1, 2026. [2]
- **Exposure reduction:** no custom Worker domain, workers.dev disabled, preview URLs disabled, routes empty.
- **Invocation ceiling:** 50 ms CPU limit configured. This is not a dollar cap.

Verification performed

All 30 tests passed after the first fix set. Six targeted billing tests passed after the second deadline fix: disabled/missing flags, cancellation without rearm, cleanup across simulated reconstruction, maximum lifetime, bot pause and stale-seat expiry. Configuration and public responses were checked. No real Cloudflare hibernation integration test has yet been completed.

Code and documentation are pushed to feat/mobile-18-holes. Worker production was deployed directly; the main checkout contains unrelated edits and was not merged. Pages frontend was not redeployed during containment. Final retired Worker version: 4a0efff6-3042-4b48-80d7-e9e065dcc18d. Owner-authorized v2-retire-rooms migration removed the namespace; production now exports only a stateless retirement response.

07 / PREVENTION AND OWNERSHIP

What remains—and what must change

Action	State	Acceptance / responsibility
Stop the unattended room loop	Closed: retired	Namespace deleted; production has no object binding.
Remove public multiplayer ingress	Verified	No Worker domain; development and preview flags false.
Fix transient lifecycle deadlines	Reference code tested	Production lifecycle removed. Real-eviction validation is required only before a future rewrite launches.
Delete ThreeWood namespace/data	Completed	Owner approved retirement. Cloudflare namespace list now contains only the two SofaPaint trackers.
Account-wide dollar spending cap	Access blocked	Billing configuration returns 403; browser session requires login. Product spend-policy availability must be verified.
Admission and message abuse controls	Closed by retirement	No multiplayer admission/message endpoint exists. Controls remain required for a future implementation.
Usage budget / automatic cutoff	Closed for retired rooms	No RoomObject execution or storage remains. Shared-account billing policies are a separate pending item.
Lifecycle telemetry and alerts	No live room lifecycle	Retirement removes the monitored workload. Account alerts remain pending authenticated access.
Real-player engagement telemetry	Deployed: static signals	Three Pages files count starts/holes/finishes without a collector Worker, database or player IDs.

Criteria for a future multiplayer rewrite

1. Use a separate staging namespace. Exercise disconnects, mobile suspension, eviction, deploys, stale sockets and empty rooms with real Cloudflare execution.
2. Show that saved deadlines survive reconstruction, no bot work continues without a genuinely live human, and expired/empty objects stop scheduling alarms.
3. Set conservative room-creation, join, socket, message-size and message-rate limits. Validate room-code collisions and admission before using short public codes.
4. Implement an aggregate usage budget with a fail-closed cutoff; estimate worst-case bot work. An external alert alone does not enforce that budget.
5. Prefer event-driven or client-hosted bots where feasible; reassess the 200 ms autonomous alarm design. Tune the 50 ms CPU limit using measured staging behavior.
6. Make restoring the public domain and enabling the flag an explicit reviewed release. Verify the kill switch under load first.

The retained reference implementation’s bot guard checks for a socket with an attached player ID. That is stronger than the old code but is not itself proof that a human is actively playing. Likewise, unit tests do not prove production hibernation behavior. The implemented fixes reduce risk; deletion of the production namespace is the current operational protection. The six old room tests remain regression evidence, not a claim that multiplayer is deployed.

08 / HANDOFF

Parked-state runbook and references

Keep while the game is inactive

Leave Pages serving solo play. Keep `main="retired.js"`, no Durable Object bindings, `workers_dev=false`, `preview_urls=false` and `routes=[]`. Preserve the completed v2-retire-rooms deletion migration. Do not reattach rooms.threewood.app or redeploy an older Worker configuration. Check the Cloudflare control plane as well as Wrangler: deployment with an empty route list did not automatically remove the old custom domain; it was explicitly removed through the API.

Retained resources and future charges

Only **sofapaint-mcp_UsageBudget** and **sofapaint-auto_UsageBudget** remain. ThreeWood's former namespace `4e52c2a9cb33472b97dcff9d1f287913` and its stored room data have been deleted. This removes future execution/storage charges for that namespace; delayed charges for earlier usage may still appear.

The shared account has 11 Worker scripts, five R2 buckets and two KV namespaces; no D1 databases or queues were listed. Daily promote-feedback and hourly sofapaint-mcp jobs remain intact. No comparable second alarm loop was found. Do not delete unrelated resources or downgrade the shared plan solely to park this game. Intentional AI requests can still incur future charges.

If charges or activity reappear

1. Check the invoice period first: delayed billing for pre-shutdown usage is not new execution.
2. Query RoomObject invocations grouped by time and type; inspect Worker domain/subdomain flags and deployed variables.
3. If new room work exists, keep ingress disconnected and inspect the live deployed version and alarm path. Avoid a frontend rollback that restores backend routes.
4. Reconcile new costs by product and namespace. Escalate with invoice and usage evidence; any billing credit is Cloudflare's decision.

Evidence and source register

Primary evidence: supplied invoice IN-87619432 (7 pages); authenticated Cloudflare namespace/domain/subdomain and Pages APIs; GraphQL invocation, periodic, AI and zone analytics; retrieved pre-containment Worker bundle; server/room.js, src/net/room.js and tests/room-billing.test.js.
Repository records: docs/cloudflare-billing-incident.md; docs/cloudflare-usage.csv. Raw investigation captures remain in /tmp/threewood-incident and should be archived securely if long-term evidence is required; /tmp is not durable storage.

[1] Durable Object pricing: developers.cloudflare.com/durable-objects/platform/pricing/
[2] Alarms and lifecycle: developers.cloudflare.com/durable-objects/api/alarms/
[3] Pages static pricing: developers.cloudflare.com/pages/functions/pricing/

Corrections to early investigation: production hosting is Cloudflare Pages, not legacy Vercel; the entire $89.21 bill was not caused by the game; objects were initially disabled, then the ThreeWood namespace was deleted by explicit owner decision; lack of multiplayer activity does not establish lack of solo play. This report supersedes those earlier ambiguities. Prepared from evidence available November 7, 2026; no continuous monitoring is implied.

09 / RETIREMENT AND SECURITY FOLLOW-UP

Closure status after owner decision

The owner chose the lowest-cost option: permanently retire multiplayer and delete its room data. That migration is deployed and verified.

Completed and verified

- Migration v2-retire-rooms deleted the RoomObject namespace and all room data. Cloudflare now lists exactly two unrelated SofaPaint namespaces.
- The deployed Worker has no service bindings, no object class, no cron and no public/custom/preview routes. It only contains a stateless 410 retirement response.
- The production Pages frontend remains static, with no Pages Functions. Multiplayer entry points and client reconnect paths default off.
- Anonymous gameplay signals use GETs for three static /play-events/ files. No collector Worker, database, cookies, seed, score or player identifier was added. Requests respect browser privacy opt-outs and offline state, deduplicate within a page and stop after 120 signals per page.
- 37 tests pass, including retirement configuration and offline-client guards. The frontend build passed; the solo-only update deployed as f29e524f, followed by static crawler-policy/404 deployment b43bfc8b.

Static scanner handling is also live: a top-level 404.html prevents arbitrary paths from serving the game. WordPress, repository and environment probes were verified returning 404. A robots.txt policy asks known AI crawlers not to index the site; it does not enforce a ban.

Access-dependent work: not falsely marked complete

The targeted scanner rule and AI crawler restriction are prepared in scripts/security/cloudflare-restrictions.json, with an idempotent apply/readback tool. Existing credentials return HTTP 403 for WAF, bot and billing settings. The browser dashboard redirects to login/security verification. These controls have **not** been applied or verified.

Current Cloudflare API documentation describes per-product Workers and Durable Objects spend policies with usage-block actions. Their availability in this account and their enforcement state have not been verified. Such policies are not a universal cap across AI, R2 and all products, and usage estimates/enforcement can lag. Nothing in this closure asserts an account-wide hard dollar cap. Budget alerts also remain pending authenticated access.

How to read engagement signals

Run scripts/security/report_engagement.py to query zone analytics by the event paths. Counts are approximate, not unique players or proof of humans: bots can request the files, opt-out/offline users are missed, and caches/analytics sampling affect visibility. HEAD/GET deployment checks may also appear. After an initial API-rate cooldown, the aggregate query succeeded and reported the known deployment-check hit, confirming path visibility without implying player activity.

This page supersedes the initial “not deleted” status. Retirement closes room-admission, room-budget and room-lifecycle risks by removing their production workload, rather than building a new metered backend. Security settings and shared-account controls still require the user’s authenticated dashboard session.

10 / SECURITY COMPLETION · 08 NOV 2026

Approved edge controls are now live

The owner approved the targeted scanner WAF rule and recognized AI-crawler block. Both are now configured and verified. Billing settings were left untouched.

Correct browser connection

The authenticated session was Windows Chrome (agent), not the separate Linux/server preview. The existing Chrome profile exposed a localhost debugging connection; its Cloudflare session allowed authorized reads and updates without extracting credentials or creating an API token. Earlier preview login failures did not mean the owner's browser was logged out.

WAF scanner restriction

Created one enabled Block rule scoped to threewood.app and www.threewood.app. It covers the approved WordPress, phpMyAdmin, XML-RPC, repository and environment-file probe paths. There were no pre-existing custom rules; none were overwritten. The saved expression, action and enabled flag matched the reviewed plan on readback.

Ruleset: `16ccb10d05314b02a5752788ee3a5a7c`
Rule: `3acfa08d69b64f13aceb509a617254df`
Reference: `threewood_block_scanner_paths_v1`

Recognized AI-crawler restriction

Set ai_bots_protection to Block and confirmed it with a separate settings read. Cloudflare automatically set the linked AI-training preference to Disallow. All other returned settings were preserved. Broad Bot Fight Mode remains off; personal AI-agent/search settings were not newly blocked. No country blocks, game-asset rate limits or paid upgrades were added.

Observed response checks

Request	Live Chrome result
https://threewood.app/	HTTP 200 · ThreeWood game title
https://threewood.app/wp-login.php	HTTP 403 · Cloudflare block-page title

The temporary check navigation was restored to the existing Cloudflare dashboard. These checks establish normal-root access and blocking of the observed probe; they are not proof of perfect scraper detection.

Remaining boundaries

No multiplayer namespace or data cleanup was repeated. Billing alerts, recipients, subscriptions and spend policies were not changed; those require separate approval. Account-wide spending protection is not claimed. Retirement remains the protection against the original room alarm loop.

This page supersedes the earlier “WAF/AI crawler controls pending” status. The invoice reconciliation and historical root-cause analysis are unchanged. Sanitized request/readback evidence is retained locally under /tmp/threewood-incident; operational details are recorded in docs/cloudflare-billing-incident.md.